Virtual Keyboard

Usability, Performance & Implementation

Research Paper Analysis & Technical Implementation

Based on Pangestu et al. (2014) & Findlater et al. (2015)

Why Virtual Keyboards Matter

Virtual keyboards have become essential input interfaces with the rise of touchscreen devices and wearables.

- **Touchscreen Dominance:** Most mobile interactions occur via touch interfaces
- **Wearable Computing:** Smartwatches and smartglasses need text input
- **Space Constraints:** Space is at a premium on diminutive screens
- **User Expectations:** Users expect familiar QWERTY layouts

> "Text entry on devices with very small screens becomes increasingly relevant and challenging."—Findlater et al. (2015)

Research Overview: Two Perspectives

Paper 1: Usability Testing

Pangestu et al. (2014)

Focus: Usability Evaluation

Investigates learnability and memorability of virtual keyboard interfaces using questionnaire-based usability testing with 68 users.

Key Question: How easily can users learn and remember virtual keyboard functions?

Paper 2: Text Entry Performance

Findlater et al. (2015)

Focus: Performance Metrics

Compares three QWERTY-based techniques (ZoomBoard, Callout, ZShift) for tiny touchscreens using controlled user studies.

Key Question: Which technique performs best for different screen sizes?

Usability Characteristics

1. Effectiveness
How accurately users accomplish goals

2. Efficiency
How quickly users perform tasks

3. Safety
Protection against accidental errors

4. Utility
Functionality meets user needs

5. Learnability
Easy for novices to accomplish tasks

6. Memorability
Easy to re-establish proficiency

Source: Pangestu et al. (2014)

User Expertise Levels

Novice Users

First-time users who understand task concepts but have low interface knowledge

Design Needs:

- Limited choices
- Informative feedback
- User manuals
- Effective tutorials

Knowledgeable Intermittent

Users with stable task concepts and wide interface knowledge, but struggle to remember structure

Design Needs:

- Consistent structure
- Clear interface
- Protection from errors
- Easy exploration

Expert Frequent

Users thoroughly familiar with both task concepts and interface interactions

Design Needs:

- Keyboard shortcuts
- Macro support
- Abbreviations
- Power user features

Usability Testing Results (Pangestu et al.)

✓ **Shift Button (93-94% success)**

Well-known by users due to familiarity with physical keyboards. Excellent learnability.

✓ **"?123" and "ABC" Buttons (97-100%)**

Clear visual cues help users understand function. Very high recognition rates.

! **"1/3", "2/3", "3/3" Buttons (Confusion)**

Poor learnability. Numbers don't mirror what will be shown on screen.

Key Insight: Buttons that mirror physical keyboard behavior or provide clear visual cues achieve better usability.

Text Entry Techniques for Tiny Screens

ZoomBoard

Keyboard zooms in when tapped, allowing character selection on enlarged view. Requires 2 taps per character.

Characteristics:

- Visual magnification
- Iterative zoom levels
- Lower error rate

Best for: Smallest screens (16mm)

Callout

Shows a callout popup above the touched key, displaying the character for confirmation before release.

Characteristics:

- Single-tap entry
- Visual feedback
- Finger position adjustable

Best for: Medium screens (24mm)

ZShift

Combines callout with zoomed view. Shows enlarged copy of occluded area with visual key feedback.

Characteristics:

- Zoomed callout
- 1-step interaction
- Lowest KSPC

Best for: Larger tiny screens (32mm)

Performance Metrics Explained

Words Per Minute (WPM)

Measures typing speed

A "word" is defined as 5 consecutive characters including spaces.

Example: 6 WPM on small screens

Keystrokes Per Character (KSPC)

Measures efficiency

Device-dependent measure. Lower is better. ZoomBoard has minimum of 2.0.

Example: Callout/ZShift achieve ~1.5

Character Error Rate (CER)

Measures accuracy

Uses Damerau-Levenshtein distance between submitted and reference text.

Example: ZoomBoard shows lower CER

Key Research Findings (Findlater et al.)

1. **Screen Size Matters**
   Performance improved as screen size increased. Different screen sizes demand different assistance types.

2. **Single-Tap Preferred**
   Users preferred techniques requiring 1-step interaction. ZoomBoard's 2-tap approach was "irritating on larger screens."

3. **Rapid Learning**
   Users quickly familiarized with all techniques. Learning effects observed across all conditions ($R^2 > 0.8$).

Design Recommendation: Use ZoomBoard for very small devices (necklaces, earrings). Use ZShift for mid-sized devices (smartwatches, bracelets).

Implementation Architecture

Framework: Kivy

Open-source Python framework for multitouch applications.
Cross-platform support.

Widget Hierarchy

KeyboardApp (App)

Main application class

KeyboardDemo (BoxLayout)

Root widget container

TextInput + VKeyboard

Text display and keyboard

Architecture Diagram

```
KeyboardApp
    |
    └── KeyboardDemo (BoxLayout)
        ├── TextInput (20%)
        └── VKeyboard (80%)
```

Key Features

- Vertical layout orientation
- Event-driven key handling
- Size hints for responsiveness

Code Walkthrough: Main Structure

Import Statements

```
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.textinput import TextInput
from kivy.uix.vkeyboard import VKeyboard
```

Widget Initialization

```
class KeyboardDemo(BoxLayout):
def __init__(self, **kwargs):
super().__init__(**kwargs)
self.orientation = "vertical"
self.text_input = TextInput(
multiline=False,
font_size=24,
size_hint=(1, 0.2)
)
self.keyboard = VKeyboard(
on_key_up=self.key_released
)
```

Application Entry Point

```
class KeyboardApp(App):
def build(self):
return KeyboardDemo()
if __name__ == "__main__":
KeyboardApp().run()
```

Key Components

BoxLayout: Vertical stacking container

TextInput: Single-line text display

VKeyboard: Built-in virtual keyboard

on_key_up: Event callback binding

Key Technical Features

Event Handler with Keycode Fix

```
def key_released(self, keyboard, keycode, *args):
# Extract character from tuple: (97, 'a') -> 'a'
key = keycode[1] if isinstance(keycode, tuple) else keycode
if key == 'backspace':
self.text_input.text = self.text_input.text[:-1]
elif key == 'spacebar':
self.text_input.text += ' '
elif len(key) == 1:
self.text_input.text += key
```

Feature Breakdown

Tuple Extraction Fix

VKeyboard returns (code, char). Extract character.

Backspace Handling

Removes last character. Supports error correction.

Spacebar Support

Explicit handling for space character.

Character Validation

len(key)==1 ensures only single chars added.

Bridging Research and Implementation

Learnability (Pangestu)
Users need familiar interfaces that leverage existing knowledge

→

Our Implementation
VKeyboard uses standard QWERTY layout familiar to all users

Memorability (Pangestu)
Shift button well-known from physical keyboards (93% success)

→

Our Implementation
VKeyboard includes standard shift/caps functionality

Error Correction (Findlater)
Users need mechanisms to fix typing mistakes efficiently

→

Our Implementation
Backspace handler removes last character with single tap

Occlusion Problem (Findlater)
Finger covers target; need visual feedback above keyboard

→

Our Implementation
TextInput displays above keyboard (Callout-style)

Lessons Learned and Recommendations

Design Lessons

Leverage Familiarity
Use QWERTY layout and standard button labels

Clear Visual Feedback
Buttons should indicate what will happen

Match Technique to Size
Different sizes need different approaches

Support Error Recovery
Easy backspace/correction mechanisms

Implementation Insights

Event-Driven Architecture
Kivy's callback system enables clean handling

Type Safety
isinstance check prevents runtime errors

Future Enhancements

- Word prediction for improved WPM
- Auto-correction for common typos
- Swipe gestures for space/delete
- Haptic feedback on key press

Conclusion

Key Takeaways

Usability First: Good design must prioritize learnability and memorability.

Context Matters: Screen size determines optimal interaction technique.

Implementation: Kivy provides effective tools for virtual keyboard apps.

Research to Practice: Academic research informs implementation decisions.

References

Pangestu, H. (2014). Usability Testing of Virtual Keyboard in Touch Screen with Questionnaire Method. CommIT, 8(1), 17-21.
DOI: 10.21512/commit.v8i1.591

Findlater, L. et al. (2015). Text Entry on Tiny QWERTY Soft Keyboards. CHI '15, Seoul, Korea.
DOI: 10.1145/2702123.2702388

Updated Implementation: Caps Lock Support

New Features

Caps Lock Toggle
Track and toggle uppercase mode

State Management
Boolean flag caps_lock_active

Character Transform
Convert to uppercase when active

Visual Feedback
Console prints ON/OFF status

Key Code Changes

```
# Track Caps Lock state
self.caps_lock_active = False
elif key == 'capslock':
self.caps_lock_active = \\
not self.caps_lock_active
elif len(key) == 1:
char = key.upper() \\
if self.caps_lock_active else key
self.text_input.text += char
```

Full Code: Imports and Class Definition

Import Statements

```
from kivy.app import App
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.textinput import TextInput
from kivy.uix.vkeyboard import VKeyboard
```

App: Base class for Kivy applications

BoxLayout: Container for vertical stacking

TextInput: Single-line text display

VKeyboard: Virtual keyboard widget

Class Definition

```
class KeyboardDemo(BoxLayout):
"""
A widget with a permanent virtual keyboard
that updates a TextInput.
Supports Caps Lock for uppercase letters.
"""
```

Inherits from BoxLayout for vertical arrangement of child widgets (TextInput + VKeyboard).

Docstring Purpose

Documents the class purpose, features, and behavior for maintainability.

Full Code: __init__ Method

Initialization Part 1

```
def __init__(self, **kwargs):
super().__init__(**kwargs)
self.orientation = "vertical"
# Track whether Caps Lock is active
self.caps_lock_active = False
```

super().__init__: Call parent class constructor

orientation: Stack children vertically

caps_lock_active: State flag for Caps Lock

Initialization Part 2

```
# ----- TextInput setup -----
self.text_input = TextInput(
multiline=False,
font_size=24,
size_hint=(1, 0.2),
hint_text="Click here..."
)
self.add_widget(self.text_input)
# ----- Virtual keyboard -----
self.keyboard = VKeyboard(
on_key_up=self.key_released
)
self.add_widget(self.keyboard)
```

multiline=False: Single line input

size_hint=(1, 0.2): 20% height

on_key_up: Bind key release event

Full Code: Key Handler Method

Method Signature and Key Extraction

```
def key_released(self, keyboard, keycode, *args):
"""Handle key releases from virtual keyboard."""
# Extract key symbol from tuple
key = keycode[1] if isinstance(
keycode, tuple) else keycode
print("Key pressed:", key)
```

keyboard: The VKeyboard instance that fired the event
keycode: Tuple (scancode, symbol) e.g., (97, 'a')
isinstance check: Handles both tuple and string formats

Caps Lock Handler

```
elif key == 'capslock':
# Toggle Caps Lock state
self.caps_lock_active = \\
not self.caps_lock_active
print(
f"Caps Lock {'ON' if ... else 'OFF'}"
)
```

Toggle: not operator flips boolean state
Feedback: Print status for debugging

Full Code: Key Handler Method

Special Keys - Backspace and Space

```
if key == 'backspace':
self.text_input.text = self.text_input.text[:-1]
elif key == 'spacebar':
self.text_input.text += ' '
```

backspace: Remove last char with slice [:-1]
spacebar: Add space character explicitly

Character Input with Caps Lock

```
elif len(key) == 1:
# Apply Caps Lock if active
char = key.upper() \\
if self.caps_lock_active else key
self.text_input.text += char
```

len(key)==1: Filter single-character keys only
.upper(): Convert to uppercase when active
+= char: Append character to text

Full Code: App Class and Entry Point

KeyboardApp Class

```
class KeyboardApp(App):
"""
Kivy application that displays
the KeyboardDemo widget.
"""
def build(self):
return KeyboardDemo()
```

App: Base class required for all Kivy applications

build(): Returns the root widget to display

KeyboardDemo(): Instantiates our custom widget

Entry Point

```
if __name__ == "__main__":
KeyboardApp().run()
```

__name__ check: Only runs when script executed directly

.run(): Starts the Kivy application event loop

How It Works

1. Python loads the script and creates KeyboardApp instance
2. .run() initializes Kivy and calls build()
3. KeyboardDemo widget tree is created and displayed
4. Event loop waits for user interaction (key presses)

Code Implementation Summary

Widget Architecture

KeyboardDemo (BoxLayout)
Root container with vertical layout

TextInput (20% height)
Single-line text display

VKeyboard (80% height)
QWERTY virtual keyboard

Event Flow

1. User taps key
VKeyboard fires on_key_up event

2. key_released callback
Processes keycode, updates text

Key Handling

Backspace
Remove last char via slicing

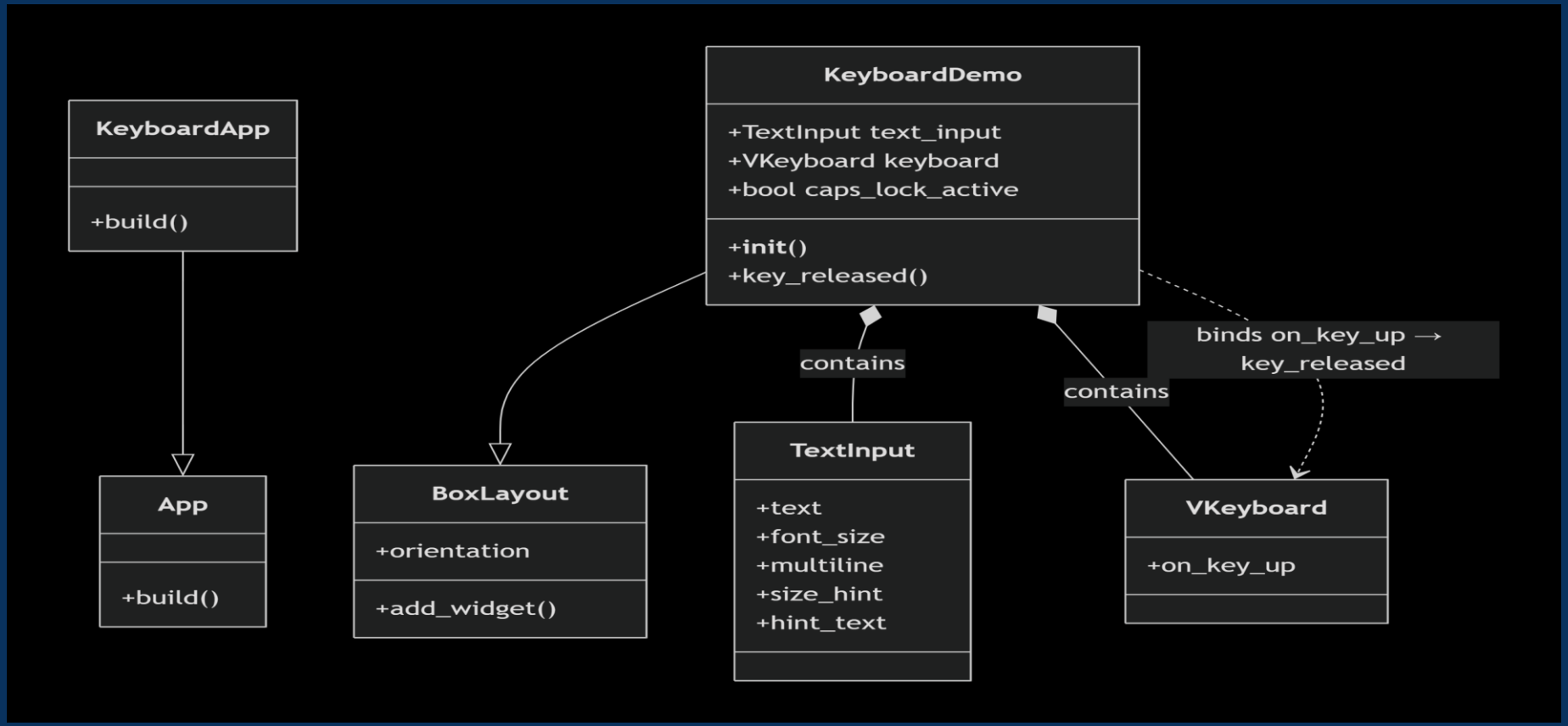
Spacebar
Append space character

Caps Lock
Toggle uppercase mode flag

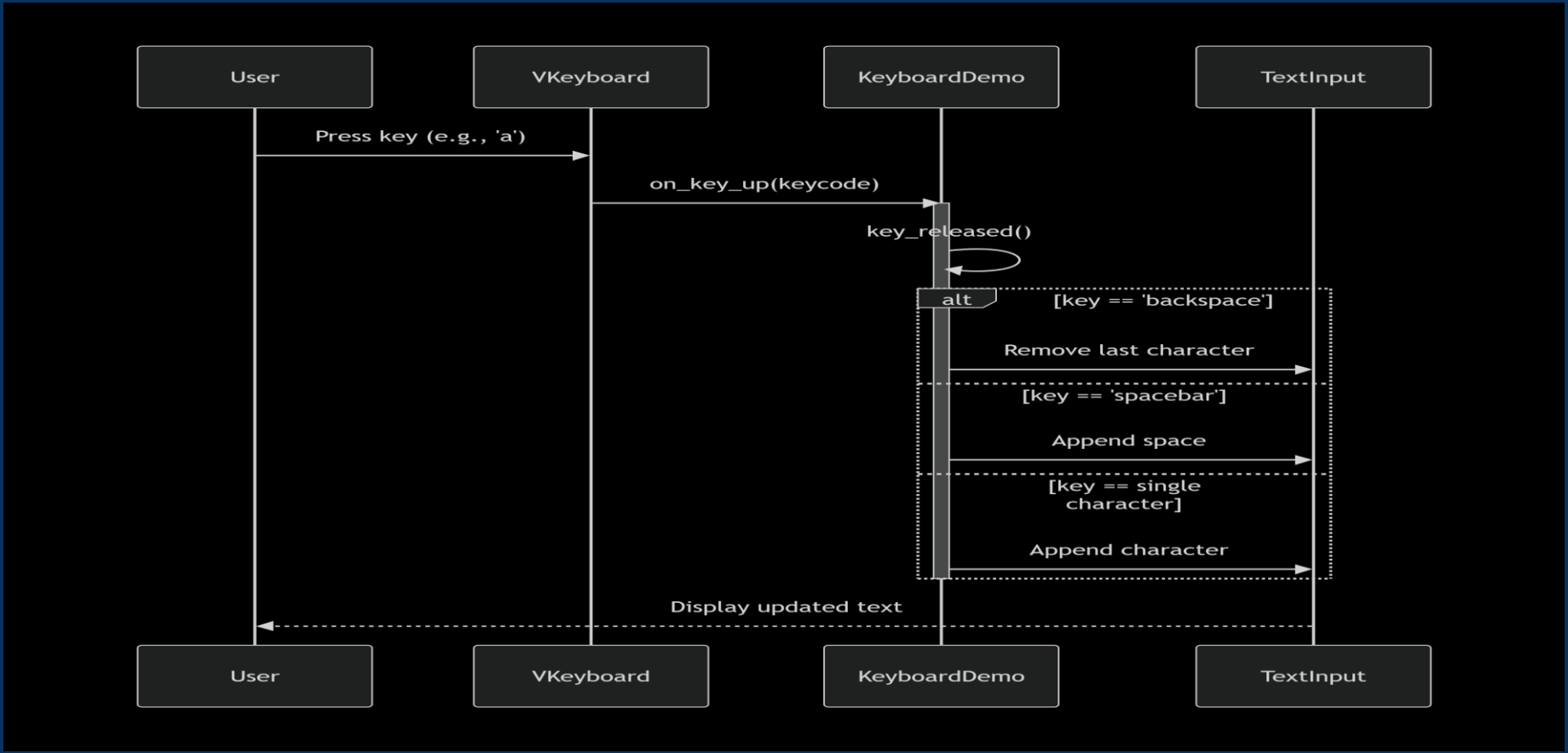
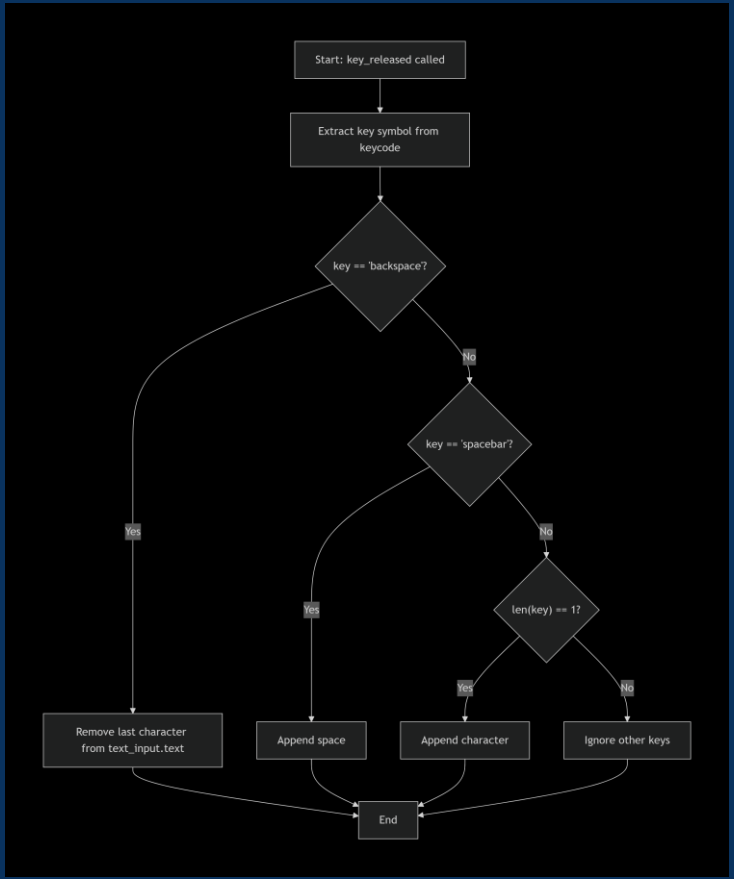
Characters
Apply uppercase if Caps Lock active

Key Fix: keycode is tuple (scancode, symbol). Use isinstance check to extract symbol safely.

Class Diagram



Sequence Diagram (Key Press Flow)



Flowchart of key_released Method